SOAP2REST Bridge

Document 05 - Backend Schema & JPA Entity Design

Database Overview

The backend uses Spring Boot 3.x, Spring Data JPA, and H2/SQLite databases. The schema supports WSDL ingestion, AI-generated mappings, dynamic REST APIs, security, analytics, and runtime proxy execution.

Table: users

id (UUID) - Primary Key
username (VARCHAR 100)
email (VARCHAR 255, UNIQUE)
password_hash (VARCHAR 500)
role (ENUM: ADMIN, DEVELOPER, VIEWER)
status (ENUM: ACTIVE, DISABLED)
created_at (TIMESTAMP)
updated_at (TIMESTAMP)

Table: api_keys

id (UUID) - Primary Key
key_name (VARCHAR 100)
api_key (VARCHAR 500)
user_id (FK → users.id)
expires_at (TIMESTAMP)
created_at (TIMESTAMP)

Table: wsdl_definitions

id (UUID) - Primary Key
service_name (VARCHAR 255)
wsdl_file_name (VARCHAR 255)
target_namespace (VARCHAR 500)
uploaded_by (FK → users.id)
upload_date (TIMESTAMP)

Table: soap_operations

id (UUID) - Primary Key
operation_name (VARCHAR 255)
wsdl_id (FK → wsdl_definitions.id)
request_schema (TEXT)
response_schema (TEXT)
created_at (TIMESTAMP)

Table: field_mappings

id (UUID) - Primary Key
operation_id (FK → soap_operations.id)
soap_field (VARCHAR 255)
rest_field (VARCHAR 255)
confidence_score (DECIMAL)
generated_by_ai (BOOLEAN)
created_at (TIMESTAMP)

Table: generated_routes

id (UUID) - Primary Key
route_name (VARCHAR 255)
http_method (VARCHAR 20)
endpoint_path (VARCHAR 500)
operation_id (FK → soap_operations.id)
status (VARCHAR 50)
created_at (TIMESTAMP)

Table: proxy_transactions

id (UUID) - Primary Key
route_id (FK → generated_routes.id)
request_payload_size (BIGINT)
response_payload_size (BIGINT)
latency_ms (BIGINT)
status_code (INTEGER)
request_time (TIMESTAMP)

Table: audit_logs

id (UUID) - Primary Key
user_id (FK → users.id)
action_type (VARCHAR 255)
description (TEXT)
created_at (TIMESTAMP)

Entity Relationships

users 1 → N api_keys
users 1 → N wsdl_definitions
wsdl_definitions 1 → N soap_operations
soap_operations 1 → N field_mappings
soap_operations 1 → N generated_routes
generated_routes 1 → N proxy_transactions
users 1 → N audit_logs

Security Model

JWT Authentication secures all dashboard APIs. API Keys secure generated REST endpoints. Passwords are stored using BCrypt hashing. Role-Based Access Control (RBAC) determines permissions.

User Roles

ADMIN: Full platform access. DEVELOPER: Upload WSDLs, generate APIs, test endpoints. VIEWER: Read-only access to analytics and documentation.

Indexes

users.email
api_keys.api_key
generated_routes.endpoint_path
proxy_transactions.request_time
soap_operations.operation_name

Sensitive Data Handling

Passwords stored using BCrypt. JWT secrets stored in environment variables. API keys encrypted before persistence. Audit logs maintained for compliance.

Backend Completion Criteria

✓ All JPA Entities Created ✓ Repository Layer Implemented ✓ Database Relationships Working ✓ JWT Authentication Integrated ✓ API Key Management Enabled ✓ Analytics Logging Operational ✓ Runtime Proxy Data Persisted